

TransitOps

Smart Transport Operations Platform

8-Hour Hackathon Build Plan | Team of 4 | Hour-by-Hour Execution & Commit Schedule

Stack: React + Node.js/Express + PostgreSQL (Prisma ORM)

1. Project Overview

TransitOps digitizes vehicle, driver, dispatch, maintenance, and expense management for logistics fleets, replacing spreadsheets/logbooks with a rules-enforcing, real-time platform.

Goal for the 8 hours

- Ship every item in §7 (Mandatory Deliverables) with correct business-rule enforcement first.
- Only after MVP is fully working end-to-end, attempt Bonus Features (§9) with remaining time.
- Every member commits to the shared repo at the end of every hour (8 commits/person, 32+ total) so integration risk stays low and progress is always demoable.

Users & Roles

Role	Primary Responsibility
Fleet Manager	Oversees fleet assets, maintenance, vehicle lifecycle, operational efficiency
Driver	Creates trips, assigns vehicles/drivers, monitors active deliveries
Safety Officer	Ensures driver compliance, tracks license validity & safety scores
Financial Analyst	Reviews expenses, fuel consumption, maintenance cost, profitability

2. Tech Stack

Layer	Choice	Why (hackathon-optimized)
Frontend	React 18 (Vite) + React Router v6	Fast HMR, minimal config, huge ecosystem
Styling	Tailwind CSS	Rapid, consistent UI without writing custom CSS
State/Data	React Context + Axios + React Query (optional)	Simple global auth state; cache-free simplicity for 8hr scope
Charts	Recharts	Fast to wire for bar/line charts (utilization, ROI, fuel efficiency)
Backend	Node.js + Express.js	Minimal boilerplate REST API, fast to scaffold
ORM	Prisma	Schema-first models, auto migrations, type-safe queries, fast seeding
Database	PostgreSQL	Single DB choice per requirement; strong constraint/enum support for status fields
Auth	JWT + bcrypt	Stateless auth, simple RBAC middleware
Validation	Zod / express-validator	Centralized request + business-rule validation
Testing/Demo	Postman collection + seed script	Fast manual verification of business rules
Version Control	Git + GitHub (feature branches → dev → main)	Parallel work, hourly commit trail, easy conflict resolution

Suggested Repository Structure

- /client — React app (Vite)
- /server — Express app
- /server/prisma/schema.prisma — DB schema & migrations
- /server/src/routes, /controllers, /middleware, /validators, /utils
- /server/prisma/seed.js — demo data for judges

- README.md — setup + demo script (owned by Member D, Hour 8)

3. Database Design — PostgreSQL

Single database engine as required: PostgreSQL. All monetary values use DECIMAL to avoid float rounding errors; all status fields are constrained via CHECK/enum.

3.1 Users

Column	Type	Constraints / Notes
id	SERIAL	PK
name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL (bcrypt)
role	VARCHAR(30)	CHECK IN (Fleet_Manager, Driver, Safety_Officer, Financial_Analyst, Admin)
created_at / updated_at	TIMESTAMP	DEFAULT now()

3.2 Vehicles

Column	Type	Constraints / Notes
id	SERIAL	PK
registration_number	VARCHAR(20)	UNIQUE, NOT NULL
name_model	VARCHAR(100)	NOT NULL
type	VARCHAR(30)	e.g. Truck, Van, Bike, Trailer
max_load_capacity_kg	DECIMAL(10,2)	NOT NULL
odometer_km	DECIMAL(10,2)	DEFAULT 0
acquisition_cost	DECIMAL(12,2)	NOT NULL
status	VARCHAR(20)	CHECK IN (Available, On Trip, In Shop, Retired), DEFAULT 'Available'
region	VARCHAR(50)	used for dashboard filter
created_at / updated_at	TIMESTAMP	DEFAULT now()

3.3 Drivers

Column	Type	Constraints / Notes
id	SERIAL	PK
name	VARCHAR(100)	NOT NULL
license_number	VARCHAR(30)	UNIQUE, NOT NULL
license_category	VARCHAR(20)	e.g. LMV, HMV
license_expiry_date	DATE	NOT NULL
contact_number	VARCHAR(20)	

Column	Type	Constraints / Notes
safety_score	DECIMAL(4,2)	DEFAULT 100
status	VARCHAR(20)	CHECK IN (Available, On Trip, Off Duty, Suspended), DEFAULT 'Available'
user_id	INT	FK → Users.id, nullable (optional login link)
created_at / updated_at	TIMESTAMP	DEFAULT now()

3.4 Trips

Column	Type	Constraints / Notes
id	SERIAL	PK
source	VARCHAR(150)	NOT NULL
destination	VARCHAR(150)	NOT NULL
vehicle_id	INT	FK → Vehicles.id, NOT NULL
driver_id	INT	FK → Drivers.id, NOT NULL
cargo_weight_kg	DECIMAL(10,2)	NOT NULL; must be ≤ vehicle.max_load_capacity_kg
planned_distance_km	DECIMAL(10,2)	NOT NULL
actual_distance_km	DECIMAL(10,2)	nullable, filled on completion
fuel_consumed_liters	DECIMAL(10,2)	nullable, filled on completion
status	VARCHAR(20)	CHECK IN (Draft, Dispatched, Completed, Cancelled), DEFAULT 'Draft'
created_by	INT	FK → Users.id
dispatched_at / completed_at	TIMESTAMP	nullable
created_at / updated_at	TIMESTAMP	DEFAULT now()

3.5 Maintenance Logs

Column	Type	Constraints / Notes
id	SERIAL	PK
vehicle_id	INT	FK → Vehicles.id, NOT NULL
maintenance_type	VARCHAR(50)	e.g. Oil Change, Tyre, Repair
description	TEXT	nullable
cost	DECIMAL(10,2)	DEFAULT 0
status	VARCHAR(20)	CHECK IN (Active, Closed), DEFAULT 'Active'
start_date	DATE	NOT NULL
end_date	DATE	nullable; set when closed
created_at / updated_at	TIMESTAMP	DEFAULT now()

3.6 Fuel Logs

Column	Type	Constraints / Notes
id	SERIAL	PK
vehicle_id	INT	FK → Vehicles.id, NOT NULL
trip_id	INT	FK → Trips.id, nullable
liters	DECIMAL(8,2)	NOT NULL
cost	DECIMAL(10,2)	NOT NULL
log_date	DATE	NOT NULL
created_at	TIMESTAMP	DEFAULT now()

3.7 Expenses

Column	Type	Constraints / Notes
id	SERIAL	PK
vehicle_id	INT	FK → Vehicles.id, nullable
trip_id	INT	FK → Trips.id, nullable
expense_type	VARCHAR(30)	CHECK IN (Toll, Maintenance, Fuel, Other)
amount	DECIMAL(10,2)	NOT NULL
expense_date	DATE	NOT NULL
description	VARCHAR(255)	nullable
created_at	TIMESTAMP	DEFAULT now()

3.8 Entity Relationships

- Users 1 — N Trips (created_by)
- Users 1 — 1 Drivers (optional login link)
- Vehicles 1 — N Trips, MaintenanceLogs, FuelLogs, Expenses
- Drivers 1 — N Trips
- Trips 1 — N FuelLogs (optional), 1 — N Expenses (optional)

4. API Endpoint Plan

Module	Endpoints
Auth	POST /api/auth/register • POST /api/auth/login • GET /api/auth/me
Vehicles	GET/POST /api/vehicles • GET/PUT/DELETE /api/vehicles/:id
Drivers	GET/POST /api/drivers • GET/PUT/DELETE /api/drivers/:id
Trips	GET/POST /api/trips • GET /api/trips/:id • PATCH /api/trips/:id/dispatch • PATCH /api/trips/:id/complete • PATCH /api/trips/:id/cancel
Maintenance	GET/POST /api/maintenance • PATCH /api/maintenance/:id/close
Fuel Logs	GET/POST /api/fuel-logs
Expenses	GET/POST /api/expenses
Dashboard	GET /api/dashboard/kpis?type=&status=®ion=

Module	Endpoints
Reports	GET /api/reports/fuel-efficiency • /utilization • /operational-cost • /roi • /export/csv

5. Business Rule Enforcement Map

Rule	Enforced In	Owner
Vehicle registration_number unique	DB UNIQUE constraint + API check	Member A
Retired / In Shop vehicles excluded from dispatch	GET /vehicles?status=Available filter used by Trip form	Member A / C
Expired-license or Suspended drivers cannot be assigned	Trip creation validator	Member B
Driver/vehicle already On Trip cannot be reassigned	Trip creation validator	Member B
Cargo weight $\leq$ max load capacity	Trip creation validator	Member A
Dispatch → vehicle & driver become On Trip	PATCH /trips/:id/dispatch transaction	Member B
Complete → vehicle & driver become Available	PATCH /trips/:id/complete transaction	Member B
Cancel (dispatched) → restores Available	PATCH /trips/:id/cancel transaction	Member B
Active maintenance → vehicle becomes In Shop	POST /maintenance transaction	Member A
Closing maintenance → Available (unless Retired)	PATCH /maintenance/:id/close transaction	Member A

Implementation note: every status-changing action (dispatch/complete/cancel trip, open/close maintenance) should run inside a single Prisma \$transaction so vehicle + driver + record updates stay atomic.

6. Team Structure & Ownership

Member	Track	Owns
Member A	Backend Lead	Auth/RBAC, DB schema & migrations, Vehicle & Driver APIs, Maintenance API
Member B	Backend Dev	Trip lifecycle & business-rule engine, Fuel/Expense APIs, Reports/Analytics calculation, CSV export
Member C	Frontend Lead	App scaffold, Auth pages, Dashboard UI, Vehicle & Driver management UI
Member D	Frontend Dev	Trip Management UI, Maintenance/Fuel/Expense UI, Analytics charts, README/demo

Pairing logic: A+B keep the backend/business-rules in lockstep each hour; C+D keep the UI consuming those exact contracts each hour — so integration happens continuously instead of at the end.

7. Hour-by-Hour Execution Plan (Commit Every Hour)

Each row = one hour. Every member pushes a commit at the end of their hour block, so the repo always reflects working, integrable progress. Suggested branch flow: each member works on their own feature branch (feat/auth, feat/trips, feat/vehicles-ui, feat/reports-ui) and merges into dev at the hourly checkpoint; dev → main after Hour 8 sanity check.

Hour 1 (0:00–1:00) — Setup & Scaffolding

Owner	Task	Commit Message
Member A	Init backend: Express app, folder structure, connect PostgreSQL, Prisma init + full schema.prisma (all 7 entities)	chore: backend scaffold + prisma schema
Member B	Draft seed script skeleton + validators folder structure; finalize ERD with Member A	chore: db seed skeleton + validators folder
Member C	Init frontend: Vite + React + Tailwind, React Router skeleton, page folder structure	chore: frontend scaffold (vite+tailwind+router)
Member D	Shared frontend utils: Axios instance with base URL/interceptors, AuthContext skeleton, base layout (Sidebar/Topbar)	chore: axios instance + auth context skeleton

Hour 2 (1:00–2:00) — Authentication & RBAC

Owner	Task	Commit Message
Member A	User model migration, POST /auth/register, POST /auth/login, JWT issuance, bcrypt hashing	feat: user auth API (register/login) with JWT
Member B	RBAC middleware (role-based route guard), auth error handling	feat: RBAC middleware
Member C	Login page UI + form validation, wire to auth API, store token	feat: login page UI integrated with auth API
Member D	ProtectedRoute wrapper + role-based nav rendering	feat: protected routes + role-based navigation

Hour 3 (2:00–3:00) — Vehicle & Driver CRUD

Owner	Task	Commit Message
Member A	Vehicle model + CRUD API, registration_number uniqueness validation	feat: vehicle CRUD APIs with uniqueness validation
Member B	Driver model + CRUD API, license-expiry helper util	feat: driver CRUD APIs with license validations
Member C	Vehicle Registry UI: list + add/edit form + status badges	feat: vehicle registry UI
Member D	Driver Management UI: list + add/edit form + status badges	feat: driver management UI

Hour 4 (3:00–4:00) — Trip Management + Core Rules

Owner	Task	Commit Message
Member A	Trip model + POST /trips with validations (cargo≤capacity, driver/vehicle availability, license/suspension check)	feat: trip creation API with validation rules
Member B	Dispatch / Complete / Cancel endpoints with atomic status transitions	feat: trip lifecycle transitions (dispatch/complete/cancel)
Member C	Trip creation form UI, dropdowns filtered to Available vehicles/drivers only	feat: trip creation form UI
Member D	Trip list UI with status filters + dispatch/complete/cancel action buttons	feat: trip list UI with lifecycle actions

Hour 5 (4:00–5:00) — Maintenance + Fuel/Expense

Owner	Task	Commit Message
Member A	Maintenance API (create/close) with automatic vehicle status switch to In Shop / restore	feat: maintenance log APIs with auto status switch
Member B	Fuel Log & Expense APIs + per-vehicle operational cost aggregation (Fuel+Maintenance)	feat: fuel & expense APIs with cost aggregation
Member C	Maintenance UI: log form + active maintenance list	feat: maintenance management UI
Member D	Fuel & Expense entry UI: forms + logs table	feat: fuel & expense logging UI

Hour 6 (5:00–6:00) — Dashboard & Reports Backend

Owner	Task	Commit Message
Member A	Dashboard KPI API: active/available vehicles, in-maintenance, active/pending trips, drivers on duty, fleet utilization %	feat: dashboard KPI aggregation API
Member B	Reports API: fuel efficiency, fleet utilization, operational cost, ROI = (Revenue-(Maint+Fuel))/AcquisitionCost	feat: analytics/reports calculation API
Member C	Dashboard UI: KPI cards + filters (vehicle type / status / region)	feat: dashboard UI with KPI cards and filters
Member D	Reports & Analytics UI: Recharts for efficiency/utilization/cost/ROI	feat: analytics charts UI

Hour 7 (6:00–7:00) — Integration, Export, Polish

Owner	Task	Commit Message
Member A	End-to-end backend integration pass; fix validation edge cases found in testing	fix: backend validation edge cases + integration fixes

Owner	Task	Commit Message
Member B	CSV export endpoint (/reports/export/csv) + global error-handling middleware	feat: CSV export API + global error handling
Member C	Full frontend integration pass: loading/error states, responsive/mobile fixes	fix: responsive UI + loading/error states
Member D	Wire CSV export button, chart polish, empty-state handling	feat: CSV export UI integration + chart polish

Hour 8 (7:00–8:00) — Final Test, Seed Data, Demo Prep

Owner	Task	Commit Message
Member A	Seed script with realistic demo data (vehicles, drivers, users); run full workflow test from §5	chore: seed demo data + workflow verification
Member B	Regression-test every business rule against the spec's Example Workflow; write brief API docs	docs: API documentation + business rule verification
Member C	Final UI polish pass (consistent spacing/colors); stretch: dark mode if time remains	style: UI polish and consistency pass
Member D	README (setup + run instructions) + demo script/slide; merge dev→main	docs: README + demo script

8. Feature Priority — Build Order

Build strictly in this order. Do not start Bonus work until every Mandatory item is demoable.

8.1 Mandatory (Hours 1–6, hardened in Hour 7)

- Responsive web interface
- Authentication with RBAC
- CRUD for Vehicles and Drivers
- Trip Management with validations
- Automatic status transitions (trip dispatch/complete/cancel, maintenance open/close)
- Maintenance workflow
- Fuel & Expense tracking
- Dashboard with KPIs (+ filters by type/status/region)
- Charts and visual analytics (fuel efficiency, utilization, cost, ROI)
- CSV export

8.2 Bonus (Only if Hour 7 finishes early / Hour 8 has slack)

- PDF export (spec marks this optional)
- Email reminders for expiring driver licenses
- Vehicle document management (upload RC/insurance docs)
- Advanced search, filters, and sorting across all tables
- Dark mode

Suggested bonus owner if time remains: Member C takes Dark Mode + Search/Sort polish; Member D takes Email Reminder stub (can be a scheduled console log / mock email service, not full SMTP, given time constraints).

9. Expected Output / Demo Flow

By the end of Hour 8, the team should be able to run this live in front of judges (mirrors spec §5 Example Workflow):

1. Log in as Fleet Manager (RBAC-protected dashboard loads with KPI cards).
2. Register vehicle 'Van-05' (max capacity 500 kg) — appears in registry, status Available.
3. Register driver 'Alex' with a valid license — appears in driver list, status Available.
4. Create a trip with cargo weight 450 kg — system validates $450 \leq 500$ and allows creation.
5. Dispatch the trip — vehicle and driver status flip to On Trip automatically; both disappear from other trip-assignment dropdowns.
6. Complete the trip (enter final odometer + fuel consumed) — vehicle and driver return to Available.
7. Create a maintenance record (e.g. Oil Change) for Van-05 — vehicle status auto-switches to In Shop and disappears from dispatch pool.
8. Open Reports — operational cost and fuel efficiency reflect the completed trip and fuel log; fleet utilization % updates on the dashboard.
9. Export the report as CSV.
10. Attempt an invalid action live (e.g. assign a Suspended driver, or 550kg cargo on Van-05) to show validation rejecting it — proves business rules, not just happy-path UI.

10. Git Workflow

- Branches: main (stable, demo-ready) ← dev (integration) ← feat/* (per member, per hour).
- Each member merges their feat branch into dev at the end of every hour — this is the "hourly commit" checkpoint.
- Quick PR self-review before merge: does it break an existing endpoint/page? If yes, fix before merging.
- At Hour 8, dev → main only after the full demo flow (§9) has been run once successfully.
- Commit message convention: feat: / fix: / chore: / docs: / style: (used throughout the table in §7).

11. Final Pre-Demo Checklist

- All 10 Mandatory Deliverables working end-to-end
- Seed data loaded so dashboard/reports aren't empty on judge screen
- .env.example committed (no real secrets in repo)
- README has exact run commands for both /client and /server
- One team member owns the live demo narration; one owns backup/local fallback in case of network issues